

U3. AP4. Ejercicios de eco concurrente

1. Servidor concurrente con thread()

Partiendo de la solución del servidor Eco TCP en modo bytes, que actualmente solo atiende a un cliente cada vez, se pide modificar el servidor para que pueda atender **múltiples clientes simultáneamente**.

El cliente **no debe modificarse**.

Para ello:

1. El servidor debe aceptar conexiones de clientes en un bucle.
2. Cada cliente conectado debe ser atendido en **un hilo independiente**.
3. Se debe crear una nueva clase llamada `ManejadorCliente` que implemente `Runnable` y se encargue de:
 - gestionar la comunicación con un cliente (streams)
 - leer bytes enviados por el cliente
 - devolver exactamente los mismos bytes (eco)
4. El servidor debe mostrar por consola:
 - cuándo un cliente se conecta
 - cuándo un cliente se desconecta
 - el **puerto remoto** del cliente
 - un **identificador numérico** único para cada cliente
5. Cuando un cliente cierre la conexión, el hilo correspondiente debe finalizar correctamente sin afectar al servidor.

El servidor debe seguir ejecutándose indefinidamente, aceptando nuevas conexiones.

Pista: El servidor pasa al manejador de cliente el socket del cliente y el id del cliente (asignado después del `accept()`).

2. Servidor TCP concurrente con límite de clientes con pool de hols

1. Requisitos del nuevo servidor

1. El servidor deberá:
 - Aceptar **múltiples clientes TCP simultáneamente**
 - Utilizar el modelo **un hilo por cliente**

- Mantener el comportamiento de **eco binario**
2. El servidor deberá limitar a **5 el número máximo de clientes atendidos simultáneamente**:
 - Cuando haya 5 clientes activos, los siguientes quedarán **en espera**
 - La gestión de los hilos deberá realizarse mediante un `ExecutorService`
 3. Como en el ejercicio anterior, se mantiene la clase `ManejadorCliente`
 - Implementa `Runnable`
 - Encargada de la comunicación con un único cliente
 4. El servidor deberá mostrar por consola:
 - Cuando un cliente se conecta
 - Cuando un cliente comienza a ser atendido (se inicia su manejador)
 - Cuando un cliente se desconecta
 5. En cada uno de estos eventos se deberá mostrar:
 - Un **identificador numérico** único del cliente
 - El **puerto remoto** desde el que se conecta
 - El número de:
 - clientes activos
 - clientes en espera

Pistas

- El límite de clientes se puede implementar mediante un `ExecutorService` con un pool fijo de hilos.
- Los contadores de clientes activos y en espera deben ser compartidos entre hilos (usar una clase `Atomic`)
- El método `accept()` no bloquea por límite de clientes.
- El servidor pasa al manejador de cliente el socket del cliente, el id del cliente (asignado después del `accept`) y referencias a los contadores de clientes activos y en espera.